

Python 语法手册

—— 配 ServerTimeDemo 项目实例

上册

语言基础 · 第一章 ~ 第五章

从词法到工程约定，逐条对照真实代码

74 个 Python 文件 · 44,490 行 · 23 张数据表

项目版本 1.9.0

手册版本 1.0

生成日期 2026 年 09 月 21 日

《Python 语法手册》全三册 · 本册为上册

本手册的每一段示例代码都摘自本项目源码，未作伪写

目录

本册是三册中的**上册**，收录 第一章 ~ 第五章。（语言基础）

第一章 手册说明与项目全景	4
1.1 怎么用这本手册（三册怎么分）	4
1.2 这个项目是什么	4
1.3 代码规模	5
1.4 分层与数据流	5
1.5 全模块地图	6
1.6 阅读约定	9
第二章 词法与基础语法	11
2.1 文件头：编码声明与模块 docstring	11
2.2 注释：为什么这么写	11
2.3 命名规范	12
2.4 缩进与续行	13
2.5 运算符速查	13
2.6 为什么这里必须用字节而不是字符	14
第三章 变量与内置类型	16
3.1 赋值与解包的各种形态	16
3.2 数字：int / float / bool	16
3.3 字符串	17
3.4 真值判定与 None	18
3.5 类型转换	19
第四章 容器类型	21
4.1 list：本项目的界面行就是它	21
4.2 tuple：固定结构与「可哈希」	22
4.3 dict：本项目的配置与行数据主力	22
4.4 set：判断「有没有」比 list 快得多	23
4.5 推导式：一行构造容器	24
4.6 排序与 key	25
4.7 可变与不可变：一个必须记住的陷阱	26

第五章 控制流	27
5.1 if / elif / else 与条件表达式	27
5.2 守卫子句：把嵌套压平	27
5.3 for 循环与它的好搭档	28
5.4 while 与 break / continue	29
5.5 海象运算符 := (3.8+)	30
5.6 match / case (3.10+)	30

第一章 手册说明与项目全景

这是一本**可以对着代码看的** Python 语法手册。它不讲「Python 是什么」，而是把语言特性一条条摆开，每条都配一段本项目（ServerTimeDemo / ExpiryManager）里正在运行的真实代码——你在项目里搜得到它、在手册里看得到它的语法依据，以及当初为什么写成那样。

这样编排的理由很实际：语法书上的例子总是干净的，而**真实的坑都长在「语法都对、组合起来就是不对」的地方**。本项目攒下的一批坑（行尾被改写、删除线画不出来、定时器断掉、目录页码不收敛）恰好都是这种，放在对应语法点旁边才记得住。

1.1 怎么用这本手册（三册怎么分）

手册按主题拆成**三册**，每册独立成书、页码各自从 1 起——想带哪本打哪本，不用把九十多页一起装订。

册	收录	主题
上册	第一章 ~ 第五章	语言基础：词法、变量与类型、容器、控制流
中册	第六章 ~ 第十章	抽象与组织：函数、类、异常、模块、注解
下册	第十一章 ~ 第十三章 + 附录 A ~ D	工程实战：标准库、GUI、工程约定；附录供速查

- **从头读**：上册 → 中册 → 下册正文，这就是语言本身的顺序（词法 → 类型 → 控制流 → 函数 → 类 → 异常 → 模块 → 注解），每一节末尾都落到项目里的一段代码。
- **当参考书查**：下册的附录 A 是一页纸的语法速查表，附录 C 是「错误写法 → 正确写法」的对照表，两处都适合直接跳过去看。
- **当项目导览读**：下册第十一章往后是「本项目真正用到的标准库」、「GUI 语法」与「工程约定与陷阱」，附录 B 是全模块清单。

版本对应关系

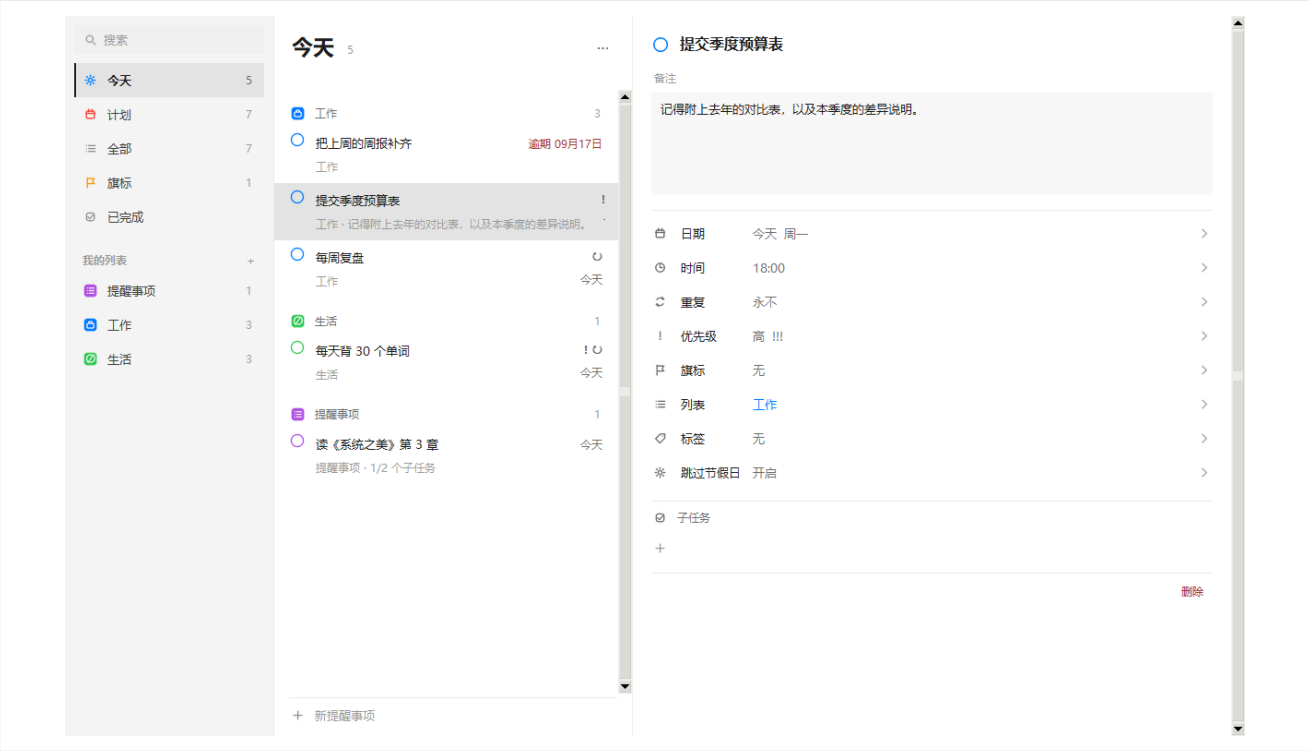
手册里引用的代码对应项目 v1.9.0。若代码后续改动，以仓库里的实际源码为准——附录 B 的模块清单与体量统计是**每次生成时现算的**，不会过时；正文里的代码是手抄的，改动后需要同步。

1.2 这个项目是什么

一句话：**一个单窗口的 Tkinter 桌面工作台**，把「服务器到期管理」「账号中心」「工具包」「学习笔记」「待办提醒」等一堆自用小功能收在一个程序里，用自绘的左侧导航切换，数据统一落在一个 SQLite 文件上。

它对学 Python 来说是个不错的样本，原因是三件事都齐了：

1. **语言面够宽**——数据类、装饰器、闭包、上下文管理、类型注解、推导式、生成器全都在用，不是玩具代码。
2. **有真实的持久化层**——23 张表的 SQLite，带迁移逻辑、事务、行工厂。
3. **有真实的界面层**——Tkinter 的坑几乎踩全了，注释里留着原因。



▲ 程序里的待办模块：左栏是清单（可拖动排序），中栏是条目列表，右栏是详情。手册里引用的界面代码大都能在这张图上找到对应形态。

1.3 代码规模

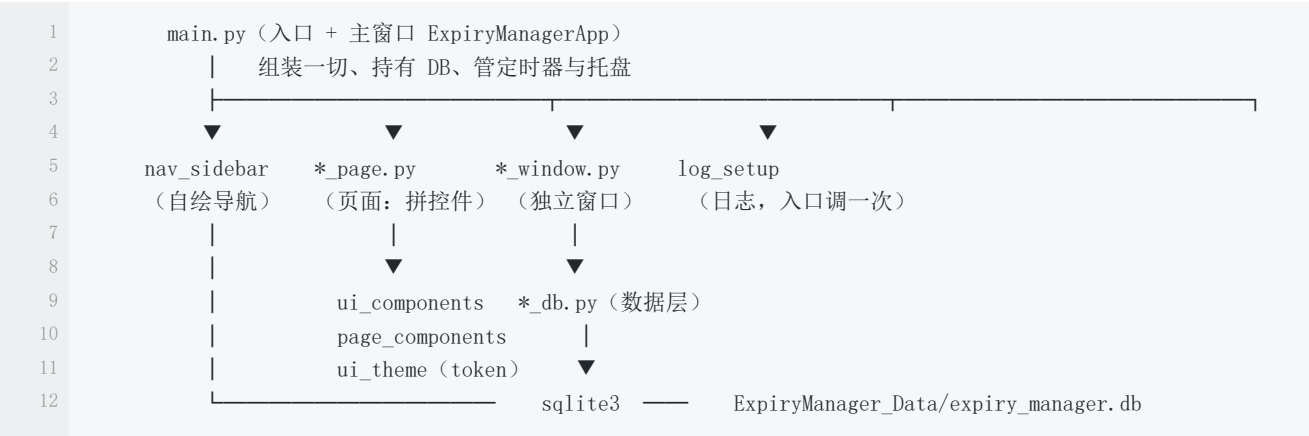
生成这份手册时统计的体量（每次生成现算，见 `scripts/make_python_manual.py`）：

位置	文件数	说明
根目录	34 个	业务模块：界面、数据层、主题、组件
embedded_admin_tools/	13 个	内置管理工具（按 services/ 分层）
scripts/	27 个	开发期工具：测试、打包核对、手册生成本身

合计 74 个 Python 文件、44,490 行。根目录里 `main.py` 一个文件就有 5,400 余行——它同时是「程序的入口」「主窗口的类定义」「一批通用工具函数」三个角色，这也是项目待办清单上「拆 `main.py`」那条的由来。

1.4 分层与数据流

模块之间的依赖是单向的，画成下来是这样：



▲ 分层约定：页面不直接写 SQL，数据层不认识 tkinter。

这条约定带来两个好处，也是项目里反复强调的：**数据层的模块可以在没有 tkinter 的解释器里跑测试**（本项目 11 个测试套件里有一批就是这么跑的），以及**换界面不用动数据逻辑**。

1.5 全模块地图

下面这张表覆盖根目录与 `embedded_admin_tools/` 的全部模块，职责取各文件自己的模块 docstring 首句 —— 想了解某个功能从哪个文件入手，查这里最快。

模块	行数	类	函数	职责
<code>main.py</code>	5444	9	39	ExpiryManager - 个人系统（到期管理器）v2.0
<code>tools_page.py</code>	4485	6	13	工具包页面（重写版：分类+包管理、相对路径、图标抽取）
<code>todo_page.py</code>	2165	3	3	待办事项模块 - 界面层（View Layer）
<code>study_notes_window.py</code>	1844	3	3	学习笔记主窗口
<code>embedded_admin_tools/api_demo_window.py</code>	1607	1	0	（无模块 docstring）
<code>scripts/test_tools_launcher.py</code>	1576	0	36	系统工具箱「启动器」回归测试。
<code>todo_db.py</code>	1545	5	10	待办事项模块 - 数据层（Data Layer）
<code>embedded_admin_tools/market_quote_window.py</code>	1458	1	0	（无模块 docstring）
<code>scripts/test_todo_ui.py</code>	1355	1	29	待办页面 UI 回归测试：真实窗口 + 样例数据 + 逐项断言。
<code>adb_page.py</code>	1080	1	0	ADB 工具箱
<code>scripts/test_todo.py</code>	976	0	17	待办 / 提醒事项模块回归测试（数据层 + 节假日引擎，纯逻辑、不碰 Tk）。
<code>embedded_admin_tools/error_code_service.py</code>	929	1	2	错误码服务 - <code>returnCode.json</code> 内嵌版本

模块	行数	类	函数	职责
<code>study_demo_window.py</code>	925	2	0	Python 学习辅助模块
<code>study_notes_db.py</code>	895	4	2	学习笔记模块 - 数据层 (Data Layer)
<code>scripts/manual_engine.py</code>	890	3	14	Python 语法手册的排版引擎 —— 把「块列表」渲染成一份带目录、页码、书签的 PDF。
<code>qa_work_log_page.py</code>	774	5	0	Q&A 问答 + 每周工作纪要 + 练习记录 页面
<code>tools_db.py</code>	768	1	38	工具库数据库
<code>qa_work_log_db.py</code>	707	8	4	Q&A 问答 + 每周工作纪要 + 练习记录 数据层
<code>qa_extras.py</code>	668	4	4	Q&A + 工作纪要 + 练习记录 扩展功能
<code>ui_theme.py</code>	592	3	0	全局 UI 主题定义。
<code>embedded_admin_tools/crypto_window.py</code>	573	1	0	(无模块 docstring)
<code>account_windows.py</code>	542	2	1	共享账户管理相关窗口。
<code>embedded_admin_tools/interface_service.py</code>	527	1	2	接口服务 - interfaces.json 内嵌版本
<code>markdown_view.py</code>	509	0	5	Markdown 渲染 (Typora 风格)
<code>expiry_dialogs.py</code>	492	3	2	到期管理相关弹窗。
<code>ui_components.py</code>	482	2	10	统一 UI 组件层。
<code>embedded_admin_tools/login_checker_window.py</code>	464	1	0	(无模块 docstring)
<code>baidu_disk_window.py</code>	463	2	0	百度网盘资料库管理窗口
<code>scripts/make_python_manual.py</code>	419	0	11	生成《Python 语法手册（配 ServerTimeDemo 项目实例）》PDF。
<code>tools_launcher.py</code>	408	0	23	<code>tools_launcher.py</code> —— 工具启动器的纯逻辑层。
<code>embedded_admin_tools/app_icon_service.py</code>	398	0	1	主程序图标内嵌服务
<code>console_page.py</code>	397	1	0	<code>console_page.py</code> - 控制台页面(日志输出 + 命令执行 + 工具维护)
<code>todo_icons.py</code>	375	0	47	待办模块的自绘图形资产。
<code>app_icons.py</code>	373	1	23	应用图形资产：应用标记 + 导航线性图标。
<code>scripts/project_audit.py</code>	367	1	10	项目静态审计脚本 — 只读分析，不修改任何被分析文件。

模块	行数	类	函数	职责
<code>scripts/prune_unused_imports.py</code>	343	0	10	未使用导入裁剪器 —— 依据 <code>pyflakes</code> 结果精确删除导入名, 只读预览为默认。
<code>nav_sidebar.py</code>	336	1	0	左侧分组导航侧栏 (自绘)。
<code>scripts/test_notes_markdown.py</code>	323	0	6	笔记页排版回归测试: Markdown 预览 tag + 编辑器外框 + 底部预览面板。
<code>scripts/test_notes_editor.py</code>	304	1	3	笔记编辑器回归测试: 格式工具栏动作 + 源码淡化 + 语法速查 + 两处预览渲染。
<code>scripts/test_detail_dialog.py</code>	288	0	7	资源详情弹窗回归测试。
<code>app_version.py</code>	285	0	3	版本号与更新日志的单一数据源。
<code>scripts/make_app_icon.py</code>	285	0	10	生成应用图标 (<code>app.ico</code>), 并同步更新内嵌 <code>base64</code> 副本。
<code>scripts/test_expiry_table.py</code>	277	0	4	到期表格「单元格收敛」回归测试。
<code>study_demo_db.py</code>	271	0	20	Python 学习模块数据库层
<code>backend_page.py</code>	244	1	0	后台首页页面类。
<code>scripts/check_name_scope.py</code>	244	0	7	名字与作用域检查 —— 只读分析, 不修改任何被检查文件。
<code>scripts/test_shell_ui.py</code>	242	0	3	外壳回归测试: 自绘侧栏 + 顶栏 + 主题 token 的运行时验证。
<code>credential_process_dialogs.py</code>	233	3	0	账号中心与流程中心相关弹窗。
<code>scripts/scan_source_writers.py</code>	215	0	3	源码写入扫描 —— 找出会「就地重写源码」的脚本 (只读分析, 绝不修改文件)。
<code>scripts/runtime_smoke.py</code>	214	0	2	运行时冒烟测试 —— 真实执行关键代码路径, 不依赖静态推断。
<code>embedded_admin_tools/login_memory_service.py</code>	212	2	1	(无模块 <code>docstring</code>)
<code>scripts/test_release_meta.py</code>	210	0	7	发布元信息回归: 版本号 / 更新日志 / <code>CHANGELOG.md</code> 是否一致。
<code>scripts/test_log_setup.py</code>	203	0	2	<code>log_setup</code> 回归测试 —— 真实写文件、真实读回, 逐条断言。
<code>credentials_page.py</code>	195	1	0	账号中心页面类。
<code>expiry_page.py</code>	185	1	5	到期管理页面类。
<code>log_setup.py</code>	176	0	5	全局日志配置 —— 在程序入口调用一次, 全项目模块自动生效。
<code>page_components.py</code>	176	0	10	页面级组件层。

模块	行数	类	函数	职责
<code>dialog_form_style.py</code>	167	0	7	弹窗表单统一样式。
<code>process_page.py</code>	152	1	0	流程中心页面类。
<code>scripts/check_exe.py</code>	147	0	2	打包产物体检：exe 时间戳 / 体积 + 从归档里反向核对源码真的进去了。
<code>scripts/window_shot.py</code>	139	1	3	把 Windows 窗口抓成 PNG —— 用 <code>PrintWindow</code> ，不依赖桌面是否可见。
<code>scripts/dep_inventory.py</code>	137	0	2	依赖清单盘点 —— 静态提取全项目 <code>import</code> ，分类为 标准库 / 第三方 / 项目内。
<code>scripts/run_all_tests.py</code>	126	0	2	一条命令跑完全部回归套件，并汇总成一张表。
<code>scripts/verify_icon_generators.py</code>	113	0	3	图标生成器等价性验证 —— 只读比对，不写入任何目标文件。
<code>scripts/gen_app_icon_service.py</code>	69	0	2	由 <code>app.ico</code> 生成内嵌图标的服务模块。
<code>scripts/gen_app_icon.py</code>	61	0	2	生成主程序图标 <code>app.ico</code> （简化版占位图）。
<code>scripts/gen_icon_service.py</code>	61	0	1	由 <code>base64</code> 文本生成内嵌图标的服务模块。
<code>embedded_admin_tools/api_client.py</code>	55	1	0	（无模块 <code>docstring</code> ）
<code>scripts/gen_changelog.py</code>	50	0	2	从 <code>app_version.VERSION_HISTORY</code> 生成仓库根目录的 <code>CHANGELOG.md</code> 。
<code>embedded_admin_tools/crypto_service.py</code>	22	1	0	（无模块 <code>docstring</code> ）
<code>embedded_admin_tools/error_code_service.py</code>	7	0	0	兼容包装：统一转发到 <code>embedded_admin_tools.services.error_code_service</code> 。
<code>embedded_admin_tools/interface_service.py</code>	7	0	0	兼容包装：统一转发到 <code>embedded_admin_tools.services.interface_service</code> 。

1.6 阅读约定

记号	含义
等宽字	代码、文件名、函数名、命令行
粗体	关键结论
▲ 出处	紧跟代码块下方，标明这段代码来自哪个文件的哪个函数
灰底方框	引自项目源码 <code>docstring</code> 的原文，是我自己当初写的注释
蓝灰色提示框	背景知识或设计取舍
橙边框警告框	踩过的坑，以及为什么不能那么写

记号	含义
绿边框结论框	拍板下来的正确做法

代码块左侧是对齐的行号，方便你在源码里定位；行号栏与代码栏底色不同，复制代码时不容易把行号一起带走。

第二章 词法与基础语法

这一章讲最小的单位：一个源文件长什么样、注释怎么写、名字怎么起、行怎么折。这些都是「一眼就能看懂、但写错了要很久才发现」的东西。

2.1 文件头：编码声明与模块 docstring

Python 3 的源文件默认就是 UTF-8，所以 `# -*- coding: utf-8 -*-` 严格来说不是必需的。项目里每个文件仍然带着它——这是延续下来的习惯，好处是文件被拿去看、被别的编辑器打开时意图明确。

真正重要的是紧跟其后的模块 **docstring**：它是文件的第一个语句，会被 `help()` 和文档工具读到。本项目把它当「文件级说明书」用，把这张表有哪些表、有哪些容易误解的语义都写在里面：

```
1 # -*- coding: utf-8 -*-
2 """
3 =====
4 待办事项模块 - 数据层 (Data Layer)
5 =====
6 本模块封装「待办 / 提醒事项」所需的全部持久化逻辑与**工作日日历**推算。
7 复用主程序的 SQLite 数据库文件 (db_path 由调用方注入)，与其余模块共享连接，
8 因此备份、迁移都只需处理一个文件。
9
10 本模块涉及的表（建表语句见 create_tables）：
11     todo_lists      清单：名称、颜色（苹果 12 色之一）、图标、排序
12     todo_items      待办条目：标题、备注、日期、时间、重复、优先级、旗标、标签
13     todo_subtasks    子任务：挂在条目下的轻量清单项
14     todo_holidays    节假日日历：放假 / 调休上班两天都记在这一张表
15     todo_state       模块内键值状态：上次工作日弹窗日期等
16
17 工作日语义（这层是「周末 / 节假日自动跳过」的全部依据）
18 =====
19 本模块把「工作日」定义为：**同时不是法定放假日、也不是周六周日**的那一天；
20 唯一的例外是**调休上班日**——它虽然落在周末，但因为被国务院安排上班，
21 所以算工作日。
22 """
```

▲ todo_db.py 开头 · 节选（原文用表格画表结构，此处为放进版心改排成列表）

docstring 的三种写法

单行："""一句话说明。""" 收在同行；**多行**：首行一句话概要、空一行、再写细节，结尾的 """ 单独一行（PEP 257）；**原始串**：内容里有反斜杠时用 `r"""..."""`，避免转义被解释。本项目的模块 docstring 统一是多行写法，因为要塞进表结构和语义约定。

2.2 注释：为什么这么写

本项目的注释有个明确取向：**不解释「这行在做什么」，只解释「为什么不是别的写法」**。代码本身已经说明前一半，后一半才是三个月后还会绊人的部分。

```
1 def _migrate_alert_columns(self):
2     """给老库补上「到点提醒」的两列。
3
4     新装的程序建表时就带上它们，但**已经装过老版本**的库里
5     ``todo_items`` 没有这两列 —— SQLite 又没有
6     ``ADD COLUMN IF NOT EXISTS``，只能先查 ``PRAGMA table_info``
7     再按需补，否则用户得删库重来。
8     """
9     existing = {
10         row["name"]
11         for row in self.conn.execute("PRAGMA table_info(todo_items)").fetchall()
12     }
13     for column, ddl in (
14         ("alerted_for", "ALTER TABLE todo_items ADD COLUMN "
15                         "alerted_for TEXT NOT NULL DEFAULT ''"),
16         ("snooze_until", "ALTER TABLE todo_items ADD COLUMN "
17                          "snooze_until TEXT NOT NULL DEFAULT ''"),
18     ):
19         if column not in existing:
20             self.conn.execute(ddl)
21     self.conn.commit()
```

▲ todo_db.py · TodoDB._migrate_alert_columns()

注意这里没有一行注释在说「查询表结构」「循环补列」。它说的是**为什么不能写** `ADD COLUMN IF NOT EXISTS`、以及**漏了会有什么后果**（用户得删库重来）。

2.3 命名规范

Python 社区只有一份约定（PEP 8），没有强制。本项目的实际用法如下，照着写就不会有突兀感：

对象	写法	项目里的例子
模块	全小写 + 下划线	todo_db、nav_sidebar、page_components
包	全小写，不用下划线	embedded_admin_tools
类	大驼峰 CapWords	TodoDB、TodoPage、ScrollArea、ManualDoc
函数 / 方法	全小写 + 下划线	pending_alerts、_migrate_alert_columns
变量	全小写 + 下划线	due_date、submit_text、moment
常量	全大写 + 下划线	ALERT_GRACE_MINUTES、TODO_TICK_MS
私有成员	前导单下划线	self._conn、_norm、_STRIKE_FONTS
类型变量	同变量	list[str]、dict[int, str]
「别碰」的成员	双下划线前后	__init__、__eq__、__enter__

单下划线与双下划线的区别

前导单下划线（`_norm`）只是「约定别在外部用」，`from m import *`会跳过它，但直接写 `m._norm()` 照样能调——它是给人的信号，不是给解释器的锁。**前导双下划线**（`self.__x`）会触发名称改写（mangling），在类 `Foo` 里变成 `_Foo__x`，用于避免与子类同名属性打架。本项目只用单下划线，因为**界面层的测试需要直接点私有方法**（例如探针里调 `page._list_drag_press(...)` 模拟拖动），双下划线反而会让测试写不下去。

2.4 缩进与续行

缩进用 **4 个空格**，项目里没有一处用 Tab。行太长时有三种合法折法，本项目三种都在用：

- 1. **括号里直接换行**（隐式续行，最常用）：`( [ {` 之内不需要任何续行符，这是首选写法，因为它不依赖行尾没有空格。
- 2. **行尾反斜杠**：只有在括号外、又必须折行时才用。项目里几乎不用——反斜杠后粘一个空格就是语法错误，而肉眼看不出来。
- 3. **把长表达式提成变量或辅助函数**：比前两种都好，本项目后来倾向这种。

```
1     for column, ddl in (  
2         ("alerted_for", "ALTER TABLE todo_items ADD COLUMN "  
3             "alerted_for TEXT NOT NULL DEFAULT ''"),  
4         ("snooze_until", "ALTER TABLE todo_items ADD COLUMN "  
5             "snooze_until TEXT NOT NULL DEFAULT ''"),  
6     ):  
7         if column not in existing:  
8             self.conn.execute(ddl)
```

▲ 括号内的隐式续行 · todo_db.py

上面这段还有一处值得留意细节：两个字符串字面量**直接相邻**（`"ALTER TABLE ... "` 和 `"alerted_for ..."`），Python 会在编译期把它们拼成一个，不会产生运行期开销——这是拆长字符串的惯用手段。

2.5 运算符速查

类别	运算符	说明 / 项目里的用法
算术	<code>+ - * /</code>	<code>/</code> 永远得到 float，即使能整除
整除 / 取余	<code>// %</code>	<code>divmod(总数, 12)</code> 一次拿到商和余（ <code>add_months()</code> ）
幂	<code>**</code>	<code>2 ** 10</code>
比较	<code>== != < <= > >=</code>	可以链式写： <code>0 <= hour <= 23</code>
身份	<code>is / is not</code>	只与 <code>None / True / False</code> 比较时用
成员	<code>in / not in</code>	<code>if column not in existing</code>
逻辑	<code>and or not</code>	短路求值； <code>not</code> 优先级最低，别省括号
位运算	<code>& ^ ~ << >></code>	本项目用在权限位与颜色通道上

类别	运算符	说明 / 项目里的用法
赋值	<code>= += -= *= /= //= %= **=</code>	还有 <code>&= =</code> 等位运算版本
三目	<code>A if 条件 else B</code>	表达式，不是语句
海象	<code>:=</code>	在表达式里赋值，见 5.6
解包	<code>*</code> / <code>**</code>	<code>f(*args, **kwargs)</code> 、 <code>a, *rest = seq</code>

`0 <= hour <= 23` 为什么在 Python 里能写

这是链式比较，Python 把它编译成 `0 <= hour and hour <= 23`，且中间的 `hour` 只求值一次。项目里解析时间就是这么校验的：`if not (0 <= hour <= 23 and 0 <= minute <= 59): return None`。

2.6 为什么这里必须用字节而不是字符

本项目有一条贯穿始终的纪律：**改源码文件必须走字节层**。先看现状——项目里不同文件的行尾是不一样的：

文件	行尾现状	成因
应用源码（ <code>main.py</code> 、 <code>todo_db.py</code> ...）	CRLF	在 Windows 上生成、被编辑器改过
<code>main.py</code>	混排：CRLF + 少量单独 LF	历史上被不同工具改过
<code>scripts/*.py</code>	LF	在 Git Bash 环境里生成
<code>*.md</code>	LF	同上

行尾混排本身不影响运行，但会把 **diff 炸掉**：如果把 `main.py` 用普通文本方式写回去，Python 的默认行为会把整个文件的行尾统一成一种，于是「改 5 行」在 `git diff` 里显示为「改了 5,000 行」。所以本项目改源码的姿势是先读进来、只替换目标片段、再原样写回：

```
1 from pathlib import Path
2
3 p = Path("todo_db.py")
4 raw = p.read_bytes()           # 字节读，不经过文本层
5 assert raw.count(b"\r\n") == raw.count(b"\n")  # 确认是纯 CRLF
6 before = raw.count(b"\n")
7 text = raw.decode("utf-8").replace("\r\n", "\n")  # 统一成 LF 再操作
8
9 # .....在这里对 text 做替换，并断言每处锚点恰好命中一次.....
10
11 out = text.replace("\n", "\r\n").encode("utf-8")
12 assert out.count(b"\n") == before + added      # 行数变化符合预期
13 assert out.count(b"\r\n") == out.count(b"\n")  # 没有意外的单独 LF
14 p.write_bytes(out)                             # 字节写，不经文本层
```

▲ 项目里改源码通用的「字节级补丁」骨架

这条纪律的三条硬要求

① 每处锚点**断言恰好命中一次**，否则整体拒写（避免半截替换）；② 断言**行尾变化量**与预期一致；③ 写完用 `py_compile` 与 `git diff --numstat` 复核变更行数。三条齐了，才敢按下执行。

第三章 变量与内置类型

Python 的变量不是「装东西的盒子」，而是**贴在对象上的名字**。理解这一点，后面很多现象（可变默认参数、闭包捕获、`is` 与 `==` 的区别）就都有了解释。

3.1 赋值与解包的各种形态

写法	效果
<code>a = b = 0</code>	两个名字指向同一个对象
<code>a, b = 1, 2</code>	元组解包，右边先构造成元组
<code>a, b = b, a</code>	交换（右边先整体求值）
<code>first, *rest = seq</code>	带星号解包， <code>rest</code> 是列表
<code>a, (b, c) = 1, (2, 3)</code>	嵌套解包
<code>x += 1</code>	对不可变对象等于 <code>x = x + 1</code> ；对列表是原地修改

项目里最常见的解包是**遍历字典 / 元组列表时当场拆开**：

```
1 REPEAT_RULES: list[tuple[str, str]] = [  
2     ("none", "永不"),  
3     ("daily", "每天"),  
4     ("weekly", "每周"),  
5     ("biweekly", "每两周"),  
6     ("monthly", "每月"),  
7     ("yearly", "每年"),  
8     ("custom", "自定"),  
9 ]  
10 REPEAT_LABELS = dict(REPEAT_RULES)           # 元组列表 -> 字典，一行搞定
```

▲ `todo_db.py` · 规则表与其反查字典同源

`dict()` 接一个「二元组序列」就能构造字典，所以下拉框要的顺序（列表）和按值查标签（字典）可以共用一份数据，永远不会不同步。

3.2 数字：int / float / bool

`int` 是任意精度的（不会溢出），`float` 是 IEEE-754 双精度（**会有误差**），`bool` 是 `int` 的子类（`True == 1`）。

```
1 def excel_serial_to_date(value: float) -> date:  
2     """Excel 的日期序列号转 date: 以 1899-12-30 为原点。"""  
3     base = datetime(1899, 12, 30)  
4     return (base + timedelta(days=float(value))).date()
```

▲ `main.py` · 导入 Excel 时的日期换算

金额和精度的规矩

`0.1 + 0.2 == 0.3` 是 **False**（结果是 `0.30000000000000004`）。所以：① 金额一律用 `int` 存「分」，或 `decimal.Decimal`；② 比较浮点用 `abs(a - b) < 1e-9`，别用 `==`；③ 展示时用 `f"{v:.2f}"` 格式化，不要指望它精确。本项目目前的金额场景都还在 Excel 导入导出环节，因此是「转成 `float` 再格式化」，没有做分单位存储——涉及记账时要改。

3.3 字符串

字符串是**不可变**的：任何「修改」都是造了一个新串。这决定了「循环里拼字符串」是 $O(n^2)$ 的坏习惯，应该先收进列表再 `join`。

三种字符串字面量

```
1 plain = "ALTER TABLE todo_items ADD COLUMN alerted_for TEXT NOT NULL DEFAULT ''"
2 fstring = f"有 {len(missed)} 条提醒已过时间: {names} {tail}"
3 raw_text = r"C:\Users\shaoy\AppData"      # r 前缀：反斜杠不转义
4 multiline = """第一行
5 第二行"""
```

▲ 三种字面量；注意 f-string 里的单引号是 SQL 里的空串，不是 Python 的

f-string 的常用写法

写法	结果
<code>f"{name}"</code>	直接插入
<code>f"{value:.2f}"</code>	保留两位小数
<code>f"{n:>3}"</code>	右对齐、宽度 3（拼菜单与行号时常用）
<code>f"{raw!r}"</code>	用 <code>repr()</code> 的结果插入，调试时看得到引号与转义
<code>f"{when:%Y-%m-%d}"</code>	直接套日期格式
<code>f"{items=}"</code>	3.8+ 的调试写法，展开成 <code>items=[1, 2]</code>

```
1 names = ", ".join(it.title for it in missed[:3])
2 tail = f" 等 {len(missed)} 条" if len(missed) > 3 else ""
3 self.tray.notify(APP_TITLE, f"有 {len(missed)} 条提醒已过时间: {names} {tail}")
```

▲ `main.py` · 托盘汇总文案；`join` 接的是生成器表达式，不需要先造列表

常用方法与切片

写法	作用
<code>s.strip()</code>	去掉两端空白（还有 <code>lstrip</code> / <code>rstrip</code> ）
<code>s.split(" ")</code>	按分隔符切成列表；不传参数则按任意空白切并去掉空串

写法	作用
<code>s.replace(a, b)</code>	全部替换
<code>s.startswith(p) / s.endswith(p)</code>	前后缀判断
<code>s.casefold()</code>	大小写无关比较用的规范化（比 <code>lower()</code> 更彻底）
<code>s.zfill(2)</code>	补零到两位，如 <code>"9".zfill(2) == "09"</code>
<code>s[:2]</code>	取前两个字符（切片不会越界报错）
<code>"-".join(seq)</code>	拼接，元素必须都是字符串
<code>s in other</code>	子串判断

```
1 def _norm(value) -> str:
2     """把任意值规范化为去空白字符串；None 视作空串。"""
3     if value is None:
4         return ""
5     return str(value).strip()
```

▲ `todo_db.py · _norm()`，全模块 30 多处在用

`_norm` 为什么值得单独存在

SQLite 取出来的列可能是 `None`、可能是 `""`、也可能是带空格的文本。如果每个字段各自 `strip()`，迟早有人漏掉一处，于是出现「明明填了日期却说没填」。收成一个函数之后，「空」在整个模块里只有一个含义：空字符串。这个函数还顺带承担了类型宽容——传进来数字也能变成字符串。

3.4 真值判定与 None

任何对象都有「真假」。下面这些**都是假**：`None`、`False`、`0`、`0.0`、`""`、`[]`、`()`、`{}`、`set()`。其余为真。

```
1 if not _norm(item.due_date) or not _norm(item.due_time):
2     return None                # 没设日期或时间 -> 这条根本不参与提醒
```

▲ `todo_db.py · alert_moment()` 的第一道闸

因为空串是假值，上面这两句既可以理解为「没设置」，也可以理解为「设置了但是空的」——两者在业务上是同一件事，正好不需要区分。

or 兜底的惯用法

```
1 color=_norm(row["color"]) or DEFAULT_LIST_COLOR,
2 repeat_rule=_norm(row["repeat_rule"]) or "none",
3 sort_order=int(row["sort_order"] or 0),
```

▲ `todo_db.py · from_row()` 里成排的兜底

`a or b` 的含义是「`a` 若为假就取 `b`」，正好用来填默认值：库里的空值 → 变成有意义的默认值。**但要注意它同时会吞掉合法的 0 与空串**——所以这里的 `int(row["sort_order"] or 0)` 是安全的（0 本来就是想要的默认值），而如果某个字段的合法值包含 0，就不能这么写。

与 None 比较：用 is

写法	结论
<code>if x is None:</code>	推荐。 <code>None</code> 是单例， <code>is</code> 是身份比较，没有重载歧义
<code>if x == None:</code>	能用但不推荐； <code>x</code> 的自定义 <code>__eq__</code> 可能改变结果
<code>if not x:</code>	判「假值」。当心它会连 0、空串、空列表一起匹配
<code>if x:</code>	判「有值」

```
1 dialog = getattr(self, "_todo_alert_win", None)
2 self._todo_alert_win = None
3 if dialog is None:
4     return
```

▲ `main.py · _close_todo_alert()`：先取出、先置空、再判断

这里刻意分了三步。如果写成 `if self._todo_alert_win: self._todo_alert_win.destroy()` 再置空，第二层调用（把引用清空后）也不会出错，但**中间任何一步抛异常都会留下一个指向已销毁控件的引用**，而 `destroy()` 之后的控件再访问会抛 `TclError: invalid command name`。先把引用置 `None`，等于让「关闭」这个操作**幂等**——重复调用安全。

3.5 类型转换

写法	结果 / 注意
<code>int("42")</code>	42; <code>int("4.2")</code> 会抛 ValueError
<code>int(4.9)</code>	4, 向零取整 ，不是四舍五入
<code>float("4.2")</code>	4.2
<code>str(42)</code>	'42'; 对对象是 <code>repr</code> 还是 <code>str</code> 取决于是否定义了 <code>__str__</code>
<code>bool([])</code>	False
<code>list("abc")</code>	['a', 'b', 'c']
<code>list({"a": 1})</code>	['a'] —— 字典转列表拿到的是 键
<code>dict [("a", 1)]</code>	{'a': 1}
<code>round(2.675, 2)</code>	2.67 ，浮点误差所致，别指望它四舍五入

`int()` 的三种失败方式

`int("")`、`int("abc")`、`int(None)` 全都抛 **ValueError** 或 **TypeError**。所以从界面或文件读来的数字一律要防：本项目用的姿势是 `int(x)` 外面套 `try / except ValueError`，或者用 `or` 先兜成默认值 —— `int(row["sort_order"] or 0)` 里的 `or 0` 就是为了让 `None` 不进 `int()`。

第四章 容器类型

四种内建容器划分得很清楚：**list 有序可变**、**tuple 有序不可变**、**dict 按键取值**、**set 只关心「有没有」**。选错的典型症状是「明明只想判断存在性，却写了一个每次都要遍历的列表」。

类型	字面量	可变	有序	适用
list	<code>[1, 2]</code>	是	是	有顺序的一批东西；界面的行、排序结果
tuple	<code>(1, 2)</code>	否	是	固定的组合；能当 dict 的键
dict	<code>{"a": 1}</code>	是	是 (3.7+ 保证插入序)	按名字取东西；配置、行数据
set	<code>{1, 2}</code>	是	否	去重、存在性判断、集合运算

注意 `{}` 是空字典，空集合要写 `set()`。

4.1 list：本项目的界面行就是它

操作	写法
追加 / 扩展	<code>items.append(x)</code> / <code>items.extend(seq)</code>
插入 / 删除	<code>items.insert(i, x)</code> / <code>del items[i]</code> / <code>items.pop()</code>
查找	<code>items.index(x)</code> (找不到抛 <code>ValueError</code>)、 <code>x in items</code>
排序	<code>items.sort()</code> (原地) vs <code>sorted(items)</code> (新列表)
倒序 / 反转	<code>items.reverse()</code> (原地)、 <code>list(reversed(items))</code>
浅复制	<code>items.copy()</code> 或 <code>items[:]</code>
统计	<code>items.count(x)</code> 、 <code>len(items)</code> 、 <code>sum(nums)</code> 、 <code>max / min</code>

```
1 due: list[TodoItem] = []
2 missed: list[TodoItem] = []
3 moments: dict[int, str] = {}
4 for row in rows:
5     item = TodoItem.from_row(row)
6     moment = alert_moment(item)
7     if moment is None or moment > now:
8         continue
9     text = moment_str(moment)
10    (missed if now - moment > grace else due).append(item)
11    moments[item.id] = text
```

▲ `todo_db.py · pending_alerts()` 内部（此处略作合并以缩短篇幅）

`(a if 条件 else b).append(x)` 这种写法能省掉一个 `if` 分支，但只在两个候选都很短时才值得。更长的情况宁可写两行——本项目在 `_run_todo_alerts` 里就是老老实实分成 `if missed:` 与 `if due:` 两段。

4.2 tuple：固定结构与「可哈希」

元组常被当成「不可变的列表」，其实它更适合表示一条结构固定的记录——比如「颜色名 + 色值」、「标签 + 个数」。它还有一个列表没有的能力：**可以当字典的键、可以放进 set**，因为它不可变、可哈希。

```
1 # 苹果提醒事项的 12 色列表色（取自 iOS 18 自建列表取色盘）
2 LIST_COLORS: list[tuple[str, str]] = [
3     ("红色", "#FF3B30"),
4     ("橙色", "#FF9500"),
5     ("黄色", "#FFCC00"),
6     ("绿色", "#34C759"),
7     ("薄荷", "#00C7BE"),
8     ("蓝色", "#007AFF"),
9     ("靛蓝", "#5856D6"),
10    ("紫色", "#AF52DE"),
11    ("玫红", "#FF2D55"),
12    ("棕色", "#A2845E"),
13    ("深灰", "#8E8E93"),
14    ("浅褐", "#D4B89F"),
15 ]
```

▲ todo_db.py · LIST_COLORS

这里为什么不用两个平铺的列表

写成 `NAMES = [...]` 加 `VALUES = [...]` 也能跑，但那样「第 3 个颜色叫什么」要靠下标对齐，插一个就会错位。**两个互相有对应关系的东西，就装进同一个元组**，让它们物理上无法分离。

4.3 dict：本项目的配置与行数据主力

操作	写法	注意
取值（带默认）	<code>d.get(k, default)</code>	键不存在也不报错； <code>d[k]</code> 会抛 <code>KeyError</code>
存在性	<code>k in d</code>	判断的是键，不是值
设默认值	<code>d.setdefault(k, v)</code>	键在就不动，不在才写入
更新多个	<code>d.update(other)</code>	或 3.9+ 的 <code>d1 d2</code> （合并出新字典）
删除	<code>del d[k]</code> / <code>d.pop(k, None)</code>	<code>pop</code> 给默认值可以不报错
遍历	<code>d.items()</code> / <code>d.keys()</code> / <code>d.values()</code>	遍历时 不能改大小
构造	<code>dict(zip(keys, vals))</code>	两个列表配成字典

```

1 def popular_tags(tools, limit: int = 0) -> list[tuple[str, int]]:
2     """统计一批工具的标签使用次数: [(标签, 个数)], 个数降序、标签升序。"""
3     counts: dict[str, int] = {}
4     label: dict[str, str] = {}
5     for tool in tools:
6         for tag in tool_tags(tool):
7             key = tag.casefold()
8             counts[key] = counts.get(key, 0) + 1
9             label.setdefault(key, tag)
10    items = [(label[key], count) for key, count in counts.items()]
11    items.sort(key=lambda item: (-item[1], item[0].casefold()))
12    return items[:limit] if limit > 0 else items

```

▲ tools_launcher.py · popular_tags()

这一小段里有四个可以直接拿去用的字典惯用法：

1. **计数**: `counts[k] = counts.get(k, 0) + 1`。不用先判断键在不在。
2. **保留首次出现的写法**: `label.setdefault(k, tag)` —— 大小写不同的标签 (`Python / python`) 只算一个, 展示时用第一次遇到的写法。
3. **两张表同源**: 计数用规范化后的键, 展示用原始写法, 各司其职。
4. `items()` **遍历**: 拿到的是 (键, 值) 二元组, 直接就地解包。

不要在遍历字典时增删键

`for k in d: del d[k]` 会抛 `RuntimeError: dictionary changed size during iteration`。正确做法是先取快照：`for k in list(d): ...`, 或者收集成列表再统一删。本项目在 `log_setup._reset_for_tests` 里就是先 `list(root.handlers)` 再逐个移除。

4.4 set: 判断「有没有」比 list 快得多

`x in list` 要逐个比对 ($O(n)$) , `x in set` 走哈希 (平均 $O(1)$) 。只要一段代码反复问「这个在里面吗」, 就该换成 **set** —— 本项目在数据库迁移里就是这么用的:

```

1     existing = {
2         row["name"]
3         for row in self.conn.execute("PRAGMA table_info(todo_items)").fetchall()
4     }
5     for column, ddl in (...):
6         if column not in existing:
7             self.conn.execute(ddl)

```

▲ todo_db.py · 先把列名收成 set, 再逐列判断

这里其实只判断了 2 次, 换不换 set 无所谓; 但同一个模式在 `main.py` 里判断 `tool_items / shared_accounts / credential_items / users` 四张表几十个列时, 就是在真省时间。统一的写法让「哪种都行」的地方也不用挑。

集合运算

写法	含义
<code>a b</code>	并集
<code>a & b</code>	交集
<code>a - b</code>	差集（在 a 不在 b）
<code>a ^ b</code>	对称差（只在一侧）
<code>a <= b</code>	a 是否为 b 的子集
<code>a.isdisjoint(b)</code>	两者是否完全不相交

4.5 推导式：一行构造容器

推导式（comprehension）是 Python 最有辨识度的语法之一。四种形态：

```
1 names = [row["name"] for row in rows]           # 列表推导
2 names = [n for n in names if n]                 # 带过滤
3 sizes = {name: len(name) for name in names}     # 字典推导
4 unique = {name.casefold() for name in names}    # 集合推导
5 total = sum(len(name) for name in names)        # 生成器表达式（不是推导）
```

▲ 四种形态

生成器表达式与列表推导只差一对方括号，但它不一次性造出列表，而是按需产出。传给 `sum / join / any / all` 这类函数时应该用生成器版本，省一次内存分配：

```
1 names = "\n".join(it.title for it in missed[:3])
```

▲ `main.py` · `join` 直接吃生成器，不需要先造列表

项目里也大量用推导式做「提取一列」的活：

```
1 headers = [normalize_text(value) for value in header_row]
2 header_map = {header: index for index, header in enumerate(headers) if header}
3 cols = [p.name for p in sorted(rows, reverse=True)[:20]]
4 labels = [item["label"] for item in parse_account_image_items(image_value)
5           if item.get("label")]
```

▲ 分别出自 `main.py` 与手册生成本身

推导式别写太长

推导式一旦超过一行、或者带两三个 `if`，可读性会迅速差过普通 `for` 循环。本项目的规矩是：**推导式里不写函数调用以外的逻辑**；需要分支就用普通循环 + `append`。判断标准很朴素 —— 「这行字能不能一口气念完」。

4.6 排序与 key

`sort()` 与 `sorted()` 都接受 `key=` 参数：**key 是一个函数，负责把元素映射成「拿来比较的东西」**。按多个条件排序，就让 key 返回元组。

```
1 scored.sort(key=lambda item: (-item[0], item[1]))
```

▲ `tools_launcher.py` · 先按分数降序、分数相同再按原顺序升序

`-item[0]` 是「降序」的常用写法（元组比较是逐项比较，负号翻转了大小关系）。第二项 `item[1]` 是原始下标，用它保证**同分时顺序稳定**，否则每次搜索结果都会自己换位置。

key 也可以是一个具名函数，条件复杂时比 lambda 清楚得多：

```
1 def sort_key(pair):
2     raw_value = pair[0]
3     number_value = self.parse_number(raw_value)
4     if number_value is not None:
5         return (0, number_value)
6     return (1, str(raw_value).lower())
7
8 items.sort(key=sort_key, reverse=reverse)
```

▲ `embedded_admin_tools/market_quote_window.py` · 表格点表头排序

这个 key 返回 `(0, 数字)` 或 `(1, 文本)`：**第一项是「能不能当数字看」的标志位**，于是所有数字行排在所有文本行前面，各自内部再比大小——表格排序里非常实用的一个小技巧。

多条件排序为什么不用连写多个 sort

Python 的 sort **稳定**，所以「先按 A 排、再按 B 排」也能得到正确结果，但那是写两次、遍历两遍。**用元组 key 一次搞定**，意图也更直白。



▲ 清单拖动重排进行中：被拖的那一行整体压暗，两行之间有一条 2px 插入指示线。松手之后算出的新顺序，最后就是写回 `todo_lists.sort_order` 的那一串值。

4.7 可变与不可变：一个必须记住的陷阱

这是 Python 最经典的坑之一，本项目在数据类那里正面撞上过：

```

1  # 错误写法：所有实例共用同一个列表！
2  @dataclass
3  class Bad:
4      tags: list[str] = []
5
6
7  # 正确写法：默认值由工厂函数「每次现造一个」
8  @dataclass
9  class TodoItem:
10     tags: list[str] = field(default_factory=list)

```

▲ `todo_db.py` · `TodoItem` 里的 `field(default_factory=list)`

根因不在 `dataclass`，在「默认值只求值一次」

函数（和方法）的默认值在定义时求值一次，之后所有调用共用同一个对象。所以 `def f(items=[])` 里的那一个空列表会被所有调用共享——往里 `append` 一次，下次调用还在。`@dataclass` 只是把这个行为原样继承了下来。判断规则很简单：默认值是可变对象（`list` / `dict` / `set`）就必须用工厂。`field(default_factory=list)` 的含义是「每次构造实例时调一次 `list()`」，于是每个实例都有自己的空列表。

不可变对象没有这个问题，所以 `TodoItem` 里几十个 `str` / `int` 字段都直接写默认值，只有 `tags` 需要 `default_factory`。

第五章 控制流

Python 的控制流语法极少，但有两种写法上的倾向值得学：**早退式（守卫子句）**与**扁平化嵌套**。项目里几乎所有稍长的函数都这么写。

5.1 if / elif / else 与条件表达式

```
1     if getattr(self, "_exiting", False):
2         return
3     try:
4         self._run_todo_alerts()
5     except Exception:
6         logger.exception("待办到点提醒巡检失败")
7     finally:
8         self.after(TODO_TICK_MS, self.todo_alert_check)
```

▲ main.py · todo_alert_check(): 先守卫、再干活、最后必续期

条件表达式（三目）是**表达式**不是语句，所以能塞进参数里：

```
1     mid_count = len(items) if self.current_scope == "completed" else len(open_items)
2     return items[:limit] if limit > 0 else items
3     "BODY" if heavy else "BODY_LIGHT"
```

▲ 三处真实用例（todo_page.py / tools_launcher.py / ui_theme.py）

为什么 Python 没有 switch（以及后来为什么又有了）

老版本里用「if / elif 链」或「字典派发」代替。本项目 main.py 里按模块 key 切换到不同页面，用的就是字典 + 显式分支，而不是上百行 elif。3.10 起有了 match 语句，但它**不是 C 的 switch**：它是**结构匹配**，能匹配序列、映射、类实例，也支持 case _: 兜底。见 5.6。

5.2 守卫子句：把嵌套压平

所谓守卫子句，就是**先把所有「不满足就早点退出」的情况写在函数开头**，让真正的主逻辑留在最外层。对比一下：

```

1  # 不推荐：条件一层套一层
2  def export(items, path):
3      if items:
4          if path:
5              if len(items) > 0:
6                  # 真正的逻辑被推到第 3 层
7                  return write(items, path)
8
9
10 # 推荐：先排除掉不成立的情况
11 def export(items, path):
12     if not items:
13         return None
14     if not path:
15         return None
16     return write(items, path)

```

本项目的 `pending_alerts` 与 `parse_moment` 都是后者：

```

1  def parse_moment(value) -> Optional[datetime]:
2      """把 ``YYYY-MM-DD HH:MM`` 解析为 datetime；非法或空值返回 None。"""
3      parts = _norm(value).replace("T", " ").split(" ")
4      if len(parts) < 2:
5          return None
6      day = parse_day(parts[0])
7      clock = parts[1].split(":")
8      if day is None or len(clock) < 2:
9          return None
10     try:
11         hour, minute = int(clock[0]), int(clock[1][:2])
12     except ValueError:
13         return None
14     if not (0 <= hour <= 23 and 0 <= minute <= 59):
15         return None
16     return datetime(day.year, day.month, day.day, hour, minute)

```

▲ `todo_db.py · parse_moment()`：五道闸，一道比一道严

手工解析而不用 `strptime` 的原因

`datetime.strptime("2026-09-18 9:05", "%Y-%m-%d %H:%M")` 里 `%H` 期望两位小时，对 `9:05` 这种**非零填充**的写法在部分实现上解析不出来。手工 `int()` 拆没有这个不确定性，而且能顺手把「小时 25」这种荒谬值一并挡掉。当输入是「用户手填 / 别的系统塞进来的」时，宁可自己拆。

5.3 for 循环与它的好搭档

写法	用途
<code>for x in seq:</code>	直接遍历元素
<code>for i, x in enumerate(seq):</code>	同时要下标（可加 <code>start=1</code> ）

写法	用途
<code>for a, b in zip(xs, ys):</code>	并行遍历两个序列（长度不齐时以短的为准）
<code>for k, v in d.items():</code>	遍历字典
<code>for line in file:</code>	逐行读文件（不要 <code>readlines</code> ）
<code>for i in range(n):</code>	按次数循环； <code>range(2, 10, 3)</code> 支持步长
<code>for x in reversed(seq):</code>	倒着来（不复制）

```

1     for column, ddl in (
2         ("alerted_for", "..."),
3         ("snooze_until", "..."),
4     ):
5         if column not in existing:
6             self.conn.execute(ddl)

```

▲ 遍历「二元组序列」时当场解包，是项目里最常用的循环形态

别在遍历列表时删除元素

`for x in items: items.remove(x)` 会**跳过元素**：删除会让后面的元素前移，而下标继续前进。正确做法是「遍历副本、删原表」（`for x in list(items): items.remove(x)`），或者直接重建（`items = [x for x in items if not 要删(x)]`）—— 后者更清楚。

5.4 while 与 break / continue

`while` 用于「不知道要循环几次」的场景。本项目里一个典型是日历推算 —— 从今天起一天天往前找，直到找到那个既不是周末、也不是法定假期的一天：

```

1     yield cur
2     cur += timedelta(days=1)

```

▲ `todo_db.py` · 工作日推算器（生成器，用 `yield` 逐个给出）

`break` 立即跳出循环，`continue` 跳到下一次迭代。两者的使用规矩是：`continue` 尽量放在循环体开头做守卫，让下半段保持平坦：

```

1     for row in rows:
2         item = TodoItem.from_row(row)
3         moment = alert_moment(item)
4         if moment is None or moment > now:
5             continue
6         text = moment_str(moment)
7         # .....下面才是真正要做的事

```

▲ `todo_db.py` · `pending_alerts()`：不合格的直接 `continue` 掉

for 的 else 分支

`for` 和 `while` 都可以带 `else`：循环自然结束时执行，被 `break` 打断时跳过。它解决的是「找了一圈没找到」这种情况，省掉一个标志位变量：

```
1 for item in items:
2     if item.id == target:
3         print("找到了")
4         break
5 else:
6     print("一圈下来没有")    # 只有没 break 过才走到这里
```

▲ 语法示例（本项目暂未使用；知道有这个写法就够）

5.5 海象运算符 := (3.8+)

它允许在表达式里赋值，主要价值是省掉一次重复计算。判断标准很明确：同一个函数调用要写两遍时才有必要用，否则只会让人多读两眼。

```
1 # 不用海象：匹配到要调用两次
2 m = pattern.search(text)
3 if m:
4     print(m.group(1))
5
6 # 用海象：调用一次
7 if (m := pattern.search(text)):
8     print(m.group(1))
```

▲ 典型用法；注意海象赋值处必须加括号

本项目基本没用到海象

因为项目的风格是「守卫子句 + 具名中间变量」——拆出来的名字本身就有可读性价值，而海象把赋值藏进条件里，反而更难扫读。知道它、但不滥用，是这一节的结论。

5.6 match / case (3.10+)

3.10 引入的 `match` 是结构模式匹配，比 `switch` 强得多：它可以按位置、按键、按类型去「拆」一个值，还支持带条件的 `case ... if ...`。

```
1 def describe(node):
2     match node:
3         case {"kind": "module", "key": key}:
4             return f"模块 {key}"
5         case {"kind": "group", "items": [first, *_]}:
6             return f"分组，第一个是 {first}"
7         case [x, y]:
8             return "二元组"
9         case _:
10            return "其他"
```

▲ 用项目里 `nav_sidebar.NAV_MODEL` 那种嵌套字典当输入会很自然（示例由本手册编写）

用不用它，先看运行环境

`match` 需要 **Python 3.10 以上**。本项目当前用 3.12 打包，语法上可用，但**没有引入**—— 因为 `match` 对这类「字典键判断」的场景并不比显式 `if node["kind"] == "module"` 更清楚，而且要照顾到「读代码的人可能不熟这个新语法」。能读懂新语法，和该不该在团队代码里用它，是两件事。